Credit Lecture R3: Sample Design, Representativeness and Class Imbalance

Deep Learning for Actuarial Modelling: a credit risk companion

AUTHOR
Mario Ervedosa

PUBLISHED
September 3, 2026

ABSTRACT

Every lecture in this series has split a sample, and none has stopped to ask how. Lecture 2 introduced the random and the out-of-time partition in nine lines, lectures 4 to 8 inherited that partition unexamined, and lecture 1 promised that class imbalance would return with force and then left the debt unpaid. This lecture pays it and audits the inheritance. It sets out the four samples a rating system is built on, derives the clustered split a panel requires, states the three rules that decide which snapshots are eligible, and then turns the machinery on the book. Four findings come out of that. The pandemic relief window is visible in the Bondora hazard as a two-month notch rather than the multi-year exclusion the industry reached for, and dropping it lifts the twelve-month probability of default by 2.4 per cent relative. The fixed-horizon table the series has used since lecture 1 cannot express a calendar exclusion at all, and the proof is that the cleaned out-of-time test sample is empty. A representativeness sweep finds a data-collection break in July 2017 that no lecture had noticed. And on the joint distribution, a classifier separates the development and out-of-time samples at an AUC of 0.96 while marginal indices call most covariates stable.

1 What the series never stopped to ask

Lecture 2 needed a test sample, so it built two. It partitioned the Bondora book at random, then partitioned it again at the 0.8 quantile of origination dates, observed that the second partition is the honest one because a lender only ever meets business written after its development window, and moved on to the generalisation loss. That passage is nine lines long and it is the whole of the series' treatment of sample design.

Everything downstream inherited it. Lectures 4 and 5 reuse the same two partitions with the same seeds, lecture 6 states that "the learning and test split is made once outside that routine", lecture 7 refits on the out-of-time learning window to show calibration failing, and lecture S3 takes the partition across into the survival table. None of them re-opens the question, because lecture 2 appeared to have settled it.

It had not, and this lecture is the audit. Sample design in credit carries at least four decisions that an insurance frequency exercise never has to make: which snapshots are eligible at all, how a partition respects a panel whose rows repeat per loan, what to do with a period during which the outcome variable stopped meaning what it usually means, and how to handle a book on which the event of interest is either commonplace or vanishingly rare depending on which portfolio is in front of you. The course's French motor third-party liability data raises none of the four.

WHERE THIS SITS IN THE SERIES

Lecture 1 defines the outcome, the twelve-month flag $D_{i,t}^{(k)}$, the availability $A_i = \tau - g_i$, the retention set $\mathcal{W}_k$ and the vintage axis. **It owns the outcome window**, and this lecture uses it rather than re-deriving it.

Lecture 2 owns the random against out-of-time contrast and the deviance results computed on it. Lecture 6 owns encoding leakage and has already measured it, so the rule that a weight of evidence table is estimated inside the learning sample belongs there. Lecture 7 owns calibration and the balance property. Lectures S1 and S3 own the person-period expansion and the mask that replaces it.

R1 owns the point-in-time question and built the calendar-indexed table that section 7 borrows. R2 owns the long-run average, the period it is taken over, and the margin of conservatism.

R3 owns the sample: which rows are eligible, how they are partitioned, whether the partition is representative, and what an imbalanced response does to the fit.

1.1 Notation

Nothing here is renamed. Lecture 1 owns the snapshot τ, the origination month g_i, the calendar identity $u = g_i + t$, the loan age t, the window flag $D_{i,t}^{(k)}$ and the availability A_i. Lecture S1 owns the exit kind and the discrete hazard.

This lecture adds the sampling apparatus of Botha and Verster (2025), which the series has not needed until now, and it adds nothing else.

Symbol	Meaning
$\mathcal{D}$	the panel dataset, one row per loan-period
$\mathcal{D}_S$	a sub-sample of $\mathcal{D}$
$\mathcal{D}_T$, $\mathcal{D}_V$	the training and validation partitions
s_f	the sampling fraction, $\lvert\mathcal{D}_T\rvert/\lvert\mathcal{D}\rvert$
N_p	the number of loans, and therefore of clusters
(i, j)	the subject-spell key: loan i, spell j
ψ	an event type, from the exit kinds S1 defines
$r_\psi(u, \mathcal{D})$	the resolution rate for event ψ in calendar month u
w_i	an observation weight

TWO SYMBOLS THAT MEAN SOMETHING ELSE IN BOTHA'S PAPERS

A reader moving between this lecture and the source papers will meet two clashes, and both were settled earlier in the series.

τ. Botha and Verster write $\tau_e(i, j)$, $\tau_r(i, j)$ and $\tau_s(i, j)$ for the entry, resolution and stopping times of a spell. Lecture 1 fixed τ as the snapshot date, so lecture S1 already declined the subscripted forms and this lecture does the same. Exit times here are durations in months on book.

T_{ij}. Botha's $T_{ij} = \tau_s - \tau_e$ is a *spell age*, whereas lecture 1's T_i is time from origination to default. The two coincide only for a first spell with no left-truncation. Bondora's snapshot gives one performance spell per loan, so the distinction does no work here, and it would do a great deal of work on a book with re-defaults.

Botha's calendar time t' is this series' u.

2 The four samples, and the fifth

A rating system is not built on a training set and a test set. It is built on four samples with four different jobs, and a fifth that only exists once the model is live.

The **development sample** fits the model. The **calibration sample** sets the level, which in the internal ratings-based world means the long-run average that lecture R2 spends half its length on, and it is generally drawn over a much longer window than the development sample because it has to span a cycle. The **out-of-sample** partition holds out facilities from the same period, so it asks whether the fit generalises across borrowers. The **out-of-time** partition holds out a later period, so it asks whether the fit generalises across time. The **application sample** is the live book the model scores, and monitoring compares it against the development sample for as long as the model is in use.

The distinction the series has never drawn is between the third and the fourth. Out-of-sample and out-of-time are not two names for a holdout. They vary different things, they fail for different reasons, and passing one is no evidence about the other. A model can generalise perfectly across borrowers within 2015 and collapse on 2019, which is exactly what lecture 7 found when the deciles all sat below the diagonal while the Gini improved.

THE REGULATION ASKS FOR BOTH, IN BLACK LETTER

Article 175 of the Capital Requirements Regulation governs the documentation of rating systems. Its fourth paragraph applies where an institution uses statistical models, and it requires the documentation to:

> *b. establish a rigorous statistical process including out-of-time and out-of-sample performance tests for validating the model;*

Read the obligation precisely, because it is narrower than a general testing mandate: the article says what the *documentation* must establish. Even read narrowly it settles the point of this section, since a documentation standard that names both tests is a standard that treats them as different tests.

Representativeness itself sits one article earlier. Article 174 lists the requirements that apply when statistical models assign exposures to grades or pools, and point (c) is the one this lecture turns on:

> *c. the data used to build the model shall be representative of the population of the institution's actual obligors or exposures;*

On the supervisory side, section 8 of the Prudential Regulation Authority's SS4/24 is titled "Data representativeness" and carries the expectations that amplify Article 174(c). Note the date: the January 2026 consolidation takes effect on 1 January 2027 alongside PS1/26, so it is the regulator's published position rather than a binding rule at the time of writing.

A caution on a citation this lecture deliberately does **not** make. Representativeness is sometimes attributed to Article 179. That article requires estimates to be representative of *long-run experience*, which is the cycle-coverage argument lecture R2 already runs, and it is a different requirement from the comparison between the development and the application population. Article 174(c) is the one that carries this lecture.

3 Three clocks, and why a random split confuses them

Lecture 2 argued that consecutive loans are not exchangeable in time, and that argument is right. It is also less than half the problem, because a credit panel carries three clocks and a random split scrambles all three at once.

The **loan age** t counts months since origination, and it is the axis a hazard runs on. The **calendar month** u counts absolute time, and it is the axis the economy runs on. The **vintage** g_i is the origination month, and it is the axis underwriting policy runs on. Lecture 1's identity $u = g_i + t$ ties them together and, on a single snapshot, ties them too tightly: hold the vintage fixed and age and calendar move together, which is why lecture 1 could not fit a point-in-time model and R1 had to rebuild the table before it could.

A random split respects none of the three. It draws loans of every vintage into both sides, so the test sample is a sample of the same underwriting policy, the same economy and the same age mix as the learning sample. That is precisely the assumption of exchangeability the book violates, and it is why lecture 2's random-split test deviance flattered the model.

An out-of-time split fixes the vintage axis and only the vintage axis. It leaves two problems standing, and this lecture is largely about them: the held-out period may be unrepresentative for reasons that have nothing to do with model quality, which is section 6, and the held-out period may differ from the development period in the covariates themselves, which is section 8.

4 The unit of a split is the loan

The moment a panel is expanded to one row per loan-month, a row-level random split becomes wrong. Lecture S1 turned 179,171 loans into roughly 2.7 million loan-months,

and a random partition of those rows would put January and February of the same loan on opposite sides of the split. The model would then be tested on a loan it had already been trained on, at a neighbouring age, with almost all of the same covariates.

The fix is to cluster before sampling. Botha and Verster (2025) state it directly:

> *In survival analysis, it is preferable to retain the entire spell history across all of its periods t_{ij} when resampling from $\mathcal{D}$, lest the subsequent survival estimates become compromised due to missing spell periods. Row-observations in $\mathcal{D}$ therefore need to be clustered around a common characteristic before sampling randomly amongst the resulting clusters. This sampling design is usually called simple random clustered sampling and implies that all monthly observations in $\mathcal{D}$ are first grouped by the loan ID i, thereby forming N_p clusters. Thereafter, we reserve a proportion ($s_f = 70\%$) thereof for $\mathcal{D}_T$, whereupon the underlying credit histories of these randomly chosen clusters are sampled into $\mathcal{D}_T$, whilst relegating the rest into $\mathcal{D}_V$.*

They put s_f in the range 60 to 80 per cent, following Siddiqi, and use 70 per cent throughout. The same design is illustrated by Baesens and co-authors.

Two consequences are worth carrying. First, the clustering has to propagate into the evaluation metric and not only into the split, which is why Botha, Verster and Scheepers (2025) build a clustered extension of the time-dependent receiver operating characteristic curve: the standard nearest-neighbour estimator assumes independent markers, and markers within a spell are not independent. Second, sub-sampling a panel is normal practice and has a second motive beyond computation, since a large enough sample drives every p-value to zero and makes significance testing meaningless.

WHERE THE SERIES ALREADY GOT THIS RIGHT, AND WHERE IT GOT LUCKY

Lecture S1's expansion is a reshaping rather than a resampling, and it never splits, so it is safe. Lecture S3 splits at the loan level before expanding, which is the correct order.

Lecture 2's partitions run on `bondora_pd.parquet`, which carries one row per loan, so a row-level split *is* a loan-level split there. The series has therefore never made this mistake, and it has never stated the rule that protects it either.

5 Which snapshots are eligible

Three rules decide whether a snapshot may enter the development sample at all, and they apply before any partition is drawn.

The outcome window must be fully observed. A flag for the k months after a reference date can only be scored where the extract covers those months, so the latest eligible reference date is the snapshot less k. Lecture 1 implements exactly this as the retention set $\mathcal{W}_k = \{i : A_i \geq k\}$, which is why the modelling table stops at July 2020 on an extract dated July 2021.

Distorted periods are excluded, with documented dates. This is section 6.

Pre-history is excluded. Snapshots predating the operational implementation of the current default definition, or of the data collection the model depends on, carry a definition that has been back-derived rather than observed. Section 8 finds an instance of this in the Bondora book that nothing in the series had noticed.

On the length of the window, the regulation is more permissive than it is usually reported to be. Article 180(2)(e) requires a retail observation period of at least five years for at least one source, and it says nothing whatever about good years and bad years or about an economic cycle. That requirement lives in the European Banking Authority's guidelines on probability of default and loss given default estimation, and attributing it to Article 180 overstates the article. What Article 180(2)(e) does say, and what is rarely quoted, is this:

> *An institution need not give equal importance to historic data if more recent data is a better predictor of loss rates.*

That sentence matters for section 6, because it means the choice is not confined to keeping a distorted period whole or dropping it whole. Weighting is available and the regulation says so.

6 The distorted window, on this book

The pandemic is the case every credit team has now argued about, and it is the right case to work because the distortion has three separate mechanisms that push the observed default rate in different directions.

Underwriting tightened, so the loans originated during the window are a different and generally better population. **Relief was granted**, so borrowers who would have rolled into 90 days past due did not, and the default flag stopped meaning what it usually means. **The economy deteriorated**, which pushes the other way. A single default flag cannot separate the three, and a model fitted across the window silently averages them.

```
import numpy as np
import polars as pl
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from scipy import stats as sps

plt.rcParams.update({
    "figure.figsize": (9, 4), "figure.dpi": 150, "font.size": 9,
    "axes.spines.top": False, "axes.spines.right": False,
})
OX, GREY, RULE, BLUE = "#8C2F1E", "#B0B0B0", "#444444", "#1F4E79"

dat  = pl.read_parquet("../data/bondora_pd.parquet")
raw  = pl.read_parquet("../data/bondora_raw.parquet",
                       columns=["LoanId", "LoanDate", "ReScheduledOn",
        "Restructured"])
surv = pl.read_parquet("../data/bondora_survival.parquet")

def month_index(col):
    """Calendar month as an integer index, matching lecture 1's u."""
    return pl.col(col).dt.year() * 12 + pl.col(col).dt.month() - 1

def as_month(u):
    return f"{u // 12}-{u % 12 + 1:02d}"

print(f"{dat.height:,} seasoned loans; {surv.height:,} loans in the
        survival table")
```

```
148,733 seasoned loans; 179,235 loans in the survival table
```

6.1 Volume and the observed rate

```
orig = (dat.with_columns(u=month_index("LoanDate"))
           .group_by("u").agg(n=pl.len(),
         dr=pl.col("default_12m").mean())
           .sort("u"))
win = orig.filter((pl.col("u") >= 2019 * 12 + 3) & (pl.col("u") <= 2020 *
        12 + 6))
print(win.with_columns(month=pl.col("u").map_elements(as_month,
        return_dtype=pl.String))
         .select("month", "n", "dr"))

pre  = orig.filter((pl.col("u") >= 2019 * 12 + 6) & (pl.col("u") <= 2020
        * 12 + 1))
post = orig.filter((pl.col("u") >= 2020 * 12 + 3) & (pl.col("u") <= 2020
        * 12 + 6))
for nm, w in (("2019-07 to 2020-02", pre), ("2020-04 to 2020-07", post)):
    print(f"{nm}: {w['n'].sum():,} loans, "
          f"weighted 12-month default rate {(w['n'] * w['dr']).sum() /
        w['n'].sum():.4f}")
```

```
shape: (16, 3)
┌─────────┬─────┬──────────┐
│ month   ┆ n   ┆ dr       │
│ ---     ┆ --- ┆ ---      │
│ str     ┆ u32 ┆ f64      │
╞═════════╪═════╪══════════╡
│ 2019-04 ┆ 3645 ┆ 0.299314 │
│ 2019-05 ┆ 3711 ┆ 0.28941  │
│ 2019-06 ┆ 3479 ┆ 0.297212 │
│ 2019-07 ┆ 5781 ┆ 0.356686 │
│ 2019-08 ┆ 5382 ┆ 0.305834 │
│ …       ┆ …   ┆ …        │
│ 2020-03 ┆ 4186 ┆ 0.262303 │
│ 2020-04 ┆ 1449 ┆ 0.132505 │
│ 2020-05 ┆ 1292 ┆ 0.119969 │
│ 2020-06 ┆ 920  ┆ 0.1      │
│ 2020-07 ┆ 693  ┆ 0.108225 │
└─────────┴─────┴──────────┘
2019-07 to 2020-02: 45,975 loans, weighted 12-month default rate 0.2800
2020-04 to 2020-07: 4,354 loans, weighted 12-month default rate 0.1181
```

Origination collapses from 7,361 loans in October 2019 to 693 in July 2020, a fall of 91 per cent. Across the eight months to February 2020 the platform wrote 45,975 loans

that went on to default at 28.00 per cent; across the four months from April 2020 it wrote 4,354 that defaulted at 11.81 per cent. A validation pack reporting the second figure as evidence of a safer book would be reporting a different book.

6.2 Relief, and what it does to the flag

The Bondora extract carries the relief directly. `Restructured` marks loans whose schedule was changed, and `ReScheduledOn` dates the change.

```
resched = (raw.filter(pl.col("ReScheduledOn").is_not_null())
              .with_columns(u=month_index("ReScheduledOn"))
              .group_by("u").agg(n=pl.len()).sort("u"))
print(f"restructured at any point: {raw['Restructured'].sum():,} of
        {raw.height:,} loans "
      f"({raw['Restructured'].mean():.1%})")
print(f"loans carrying a rescheduling date: {resched['n'].sum():,}")

base = resched.filter((pl.col("u") >= 2019 * 12 + 8) & (pl.col("u") <=
        2020 * 12 + 1))["n"].mean()
apr  = resched.filter(pl.col("u") == 2020 * 12 + 3)["n"][0]
print(f"baseline September 2019 to February 2020: {base:,.0f} a month")
print(f"April 2020: {apr:,}, a factor of {apr / base:.2f}")
```

```
restructured at any point: 56,008 of 179,235 loans (31.2%)
loans carrying a rescheduling date: 63,044
baseline September 2019 to February 2020: 1,597 a month
April 2020: 3,076, a factor of 1.93
```

Nearly a third of the book was restructured at some point, and the monthly rescheduling count doubles in April 2020 against its immediately preceding baseline. Consequently the flag that lecture 1 built the outcome on was, for those borrowers, recording an administrative decision as much as a credit event. That is the mechanism the Bank for International Settlements' Financial Stability Institute described across jurisdictions in May 2020: mass payment deferral broke the mapping between the past-due signal and its usual meaning, so a model reading the signal at face value reads it wrongly.

6.3 The notch

The clearest view of the distortion is the monthly hazard, taken from the survival table rather than the fixed-horizon one, because the hazard is indexed by calendar month and can therefore show a month.

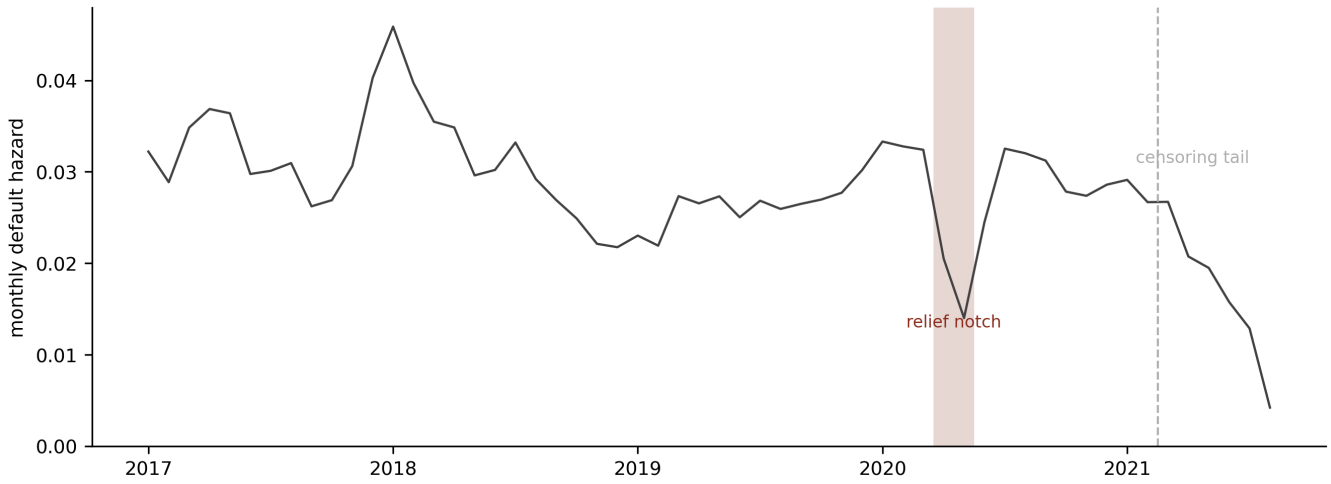
```
HORIZON = 60
COUNTRIES = ["EE", "FI", "ES"]           # R1's restriction: thick
        monthly risk sets only

pp = (surv.filter(pl.col("Country").is_in(COUNTRIES))
          .with_columns(n_periods=pl.min_horizontal(pl.col("t_obs_m"),
        pl.lit(HORIZON)))
          .with_columns(t=pl.int_ranges(1, pl.col("n_periods") + 1))
          .explode("t")
          .with_columns(
              y=((pl.col("t") == pl.col("t_obs_m"))
                 & (pl.col("exit_kind") == "default")).cast(pl.Int8),
              u=month_index("LoanDate") + pl.col("t")))
print(f"{pp.height:,} loan-months, {pp['y'].sum():,} default events")

haz = (pp.group_by("u").agg(n=pl.len(), d=pl.col("y").sum()).sort("u")
         .with_columns(q=pl.col("d") / pl.col("n"))
         .filter(pl.col("n") >= 500))
show = haz.filter((pl.col("u") >= 2019 * 12 + 5) & (pl.col("u") <= 2021 *
        12 + 5))
print(show.with_columns(month=pl.col("u").map_elements(as_month,
        return_dtype=pl.String))
          .select("month", "n", "q"))
normal = haz.filter((pl.col("u") >= 2018 * 12) & (pl.col("u") <= 2020 *
        12 + 1))
print(f"mean hazard January 2018 to February 2020:
        {normal['q'].mean():.5f}")
```

```
2,668,838 loan-months, 70,939 default events
shape: (25, 3)
```

month	n	q
str	u32	f64
2019-06	41801	0.025023
2019-07	43602	0.026834
2019-08	47503	0.025935
2019-09	50892	0.026487
2019-10	54372	0.026962
…	…	…
2021-02	59234	0.026674
2021-03	59078	0.02671
2021-04	59123	0.020736
2021-05	60297	0.019487
2021-06	62003	0.015757

```
└──────────┴──────────┴──────────┘
mean hazard January 2018 to February 2020: 0.02904
```

```
h = haz.filter(pl.col("u") >= 2017 * 12)
fig, ax = plt.subplots(figsize=(9, 3.4))
ax.plot(h["u"], h["q"], color=RULE, lw=1.1)
ax.axvspan(2020 * 12 + 2.5, 2020 * 12 + 4.5, color=OX, alpha=0.18,
        linewidth=0)
ax.axvline(2021 * 12 + 1.5, color=GREY, lw=1, ls="--")
ax.set_ylim(0, None)
ax.set_ylabel("monthly default hazard")
ticks = [y * 12 for y in range(2017, 2022)]
ax.set_xticks(ticks); ax.set_xticklabels([str(y) for y in range(2017,
        2022)])
ax.annotate("relief notch", (2020 * 12 + 3.5, 0.013), ha="center",
        fontsize=8, color=OX)
ax.annotate("censoring tail", (2021 * 12 + 3.2, 0.031), ha="center",
        fontsize=8, color=GREY)
plt.tight_layout(); plt.show()
```



Monthly default hazard on the Estonian, Finnish and Spanish sub-book, restricted to calendar months carrying at least 500 loans at risk. The hazard runs a little under 3 per cent for two years, falls to 1.40 per cent in May 2020 and returns to 3.25 per cent by July. The shaded pair of months is the relief notch that section 7 excludes. The decline from March 2021 is a different artefact, namely the thinning of observed exits as the July 2021 extract date approaches, and it is excluded from both fits rather than treated as economics.

The notch is two months wide. Against a mean of 2.90 per cent over the twenty-six months to February 2020, the hazard reads 3.24 per cent in March 2020, 2.05 in April, 1.40 in May, 2.45 in June and 3.25 in July. May 2020 is 57 per cent below the March reading, and the book is back to normal within eight weeks.

That shape is worth dwelling on, because it is not the shape the industry assumed. The distortion is sharp, dated and short, and it coincides to the month with the rescheduling spike. An exclusion window of two or three years, which was a common answer, would discard several years of usable experience to remove eight weeks of contamination, and on a book with only one other stress episode that trade is expensive. Lecture R2 declined to exclude the pandemic year for exactly that reason, and this section supplies the evidence R2 asserted.

7 What an exclusion costs, and the table that cannot pay it

Having dated the distortion, the obvious next step is to drop it and refit. The obvious next step is impossible on the table this series has used since lecture 1, and the impossibility is more instructive than the refit.

7.1 Why the fixed-horizon table cannot express a calendar exclusion

`bondora_pd.parquet` carries one row per loan, and the outcome window opens at origination. A loan originated in month g therefore carries performance over $[g, g+12]$. Excluding a distorted calendar window means excluding every loan whose window *overlaps* it, and the arithmetic of that is brutal.

```
d = dat.sort("LoanDate")
cut = d["LoanDate"].quantile(0.8)
oot = d.filter(pl.col("LoanDate") >= cut)
oot_lo, oot_hi = oot["LoanDate"].min(), oot["LoanDate"].max()
print(f"out-of-time test sample: {oot.height:,} loans, {oot_lo} to
        {oot_hi}")

import datetime as dt
for start in (dt.date(2020, 3, 1), dt.date(2020, 4, 1)):
    overlap_cut = dt.date(start.year - 1, start.month, start.day)   # g +
        12m >= start
    survivors = oot.filter(pl.col("LoanDate") < overlap_cut).height
    naive = oot.filter(pl.col("LoanDate") < start)
    print(f"distortion from {start}: overlap-based exclusion drops g >=
        {overlap_cut}, "
          f"leaving {survivors} of {oot.height:,} test loans")
    if naive.height:
        closes = dt.date(oot_hi.year + 1, oot_hi.month, oot_hi.day)
        print(f"    origination-based exclusion leaves {naive.height:,},
        whose windows "
              f"close between {start} and {closes}")
```

```
out-of-time test sample: 29,874 loans, 2019-11-04 to 2020-07-19
distortion from 2020-03-01: overlap-based exclusion drops g >= 2019-03-
01, leaving 0 of 29,874 test loans
    origination-based exclusion leaves 21,334, whose windows close
between 2020-03-01 and 2021-07-19
distortion from 2020-04-01: overlap-based exclusion drops g >= 2019-04-
01, leaving 0 of 29,874 test loans
    origination-based exclusion leaves 25,520, whose windows close
between 2020-04-01 and 2021-07-19
```

The cleaned out-of-time test sample is empty, and the count is exactly zero rather than merely small. Excluding a distortion that begins in March 2020 removes every loan originated from March 2019, whereas the out-of-time window only opens in November 2019, so the entire test sample goes. Excluding on the origination date alone leaves 21,334 loans, and it is a false comfort, because every one of their outcome windows closes inside the pandemic.

This is the same limitation lecture R2 recorded when it noted that a cohort defined by origination vintage cannot show grade migration. A table indexed by origination can

answer questions about vintages and cannot answer questions about calendar months, and a relief window is a question about calendar months.

7.2 The demonstration, on the table that can

R1 rebuilt the book as person-periods indexed by calendar month for exactly this reason. In that layout the risk set is re-formed every month, so a month can simply be dropped, and the loans that lived through it keep the rest of their history.

```
DISTORTED = [2020 * 12 + 3, 2020 * 12 + 4]      # April and May 2020
TAIL      = 2021 * 12 + 2                        # censoring tail,
        dropped from both fits
BINS      = [0, 3, 6, 12, 18, 24, 36, 60]
LBL       = [f"age{lo + 1}_{hi}" for lo, hi in zip(BINS[:-1], BINS[1:])]

panel = pp.filter(pl.col("u") < TAIL)
dropped = panel.filter(pl.col("u").is_in(DISTORTED)).height
print(f"loan-months in scope {panel.height:,}; "
      f"dropping April and May 2020 removes {dropped:,} ({dropped /
        panel.height:.2%})")

def cells(df):
    return (df.group_by(["t", "Country", "Rating"])
              .agg(n=pl.len(), d=pl.col("y").sum())
              .sort(["t", "Country", "Rating"]).to_pandas())

def design(c, columns=None):
    age = pd.cut(c["t"], BINS, labels=LBL)
    X = pd.get_dummies(pd.DataFrame({"Country": c["Country"], "Rating":
        c["Rating"],
                                     "age": age}),
        drop_first=True).astype(float)
    X = sm.add_constant(X, has_constant="add")
    return X if columns is None else X.reindex(columns=columns,
        fill_value=0.0)

c_full  = cells(panel)
c_clean = cells(panel.filter(~pl.col("u").is_in(DISTORTED)))
X_full  = design(c_full)
X_clean = design(c_clean, X_full.columns)

def glm(c, X):
    return sm.GLM(np.c_[c["d"], c["n"] - c["d"]], X,
                  family=sm.families.Binomial()).fit()

m_full, m_clean = glm(c_full, X_full), glm(c_clean, X_clean)
print(f"fitted on {len(c_full):,} and {len(c_clean):,} cells")
```

```
loan-months in scope 2,322,666; dropping April and May 2020 removes
138,264 (5.95%)
fitted on 1,382 and 1,381 cells
```

Both fits share one specification, namely a baseline hazard in loan-age bands plus country and credit rating. Only the months in the risk set differ, so any movement between them is attributable to the exclusion alone. The quantity to compare is the object a lender actually uses, which is the twelve-month cumulative probability of default implied by the fitted hazards.

```
REF_RATING = c_full.groupby("Rating")["n"].sum().idxmax()

def term_structure(model):
    """Twelve monthly hazards and the cumulative PD for the reference
       profile."""
    q = []
    for t in range(1, 13):
        row = pd.Series(0.0, index=X_full.columns)
        row["const"] = 1.0
        if f"Rating_{REF_RATING}" in row.index:
            row[f"Rating_{REF_RATING}"] = 1.0
        for lo, hi, nm in zip(BINS[:-1], BINS[1:], LBL):
            if f"age_{nm}" in row.index and lo < t <= hi:
                row[f"age_{nm}"] = 1.0
        q.append(float(model.predict(pd.DataFrame([row]))[0]))
    q = np.array(q)
    return q, 1 - np.prod(1 - q)

q_full,  pd_full  = term_structure(m_full)
q_clean, pd_clean = term_structure(m_clean)
print(f"reference profile: Estonia, rating {REF_RATING}")
print(f"12-month PD, distorted months retained : {pd_full:.4f}")
print(f"12-month PD, distorted months excluded : {pd_clean:.4f}")
print(f"difference {pd_clean - pd_full:+.4f}, "
      f"{(pd_clean / pd_full - 1) * 100:+.2f} per cent relative")
```

```
reference profile: Estonia, rating C
12-month PD, distorted months retained : 0.1820
12-month PD, distorted months excluded : 0.1863
difference +0.0043, +2.37 per cent relative
```

Excluding 5.95 per cent of the loan-months raises the estimated twelve-month probability of default from 18.20 to 18.63 per cent, a relative increase of 2.37 per cent. The direction is the one a supervisor cares about: retaining the relief window

understates risk, because relief suppressed defaults that the borrower's circumstances would otherwise have produced.

The magnitude deserves an honest reading in both directions. On the one hand 2.37 per cent of a probability of default is not nothing, and it flows straight through to a capital number by lecture R2's machinery. On the other hand it is a good deal smaller than the rhetoric around pandemic exclusions implied, and it was bought by dropping two months rather than two years. A firm that excluded 2020 through 2022 wholesale paid several years of cycle coverage for a correction of this size, and lecture R2 already showed that cycle coverage is the scarcer commodity on this book.

8 Is the held-out sample representative?

An out-of-time partition is a claim as well as a test. The claim is that the held-out period stands in for the business the model will actually meet, and Article 174(c) asks the firm to show it. This section runs the standard battery on the split the series has been using since lecture 2, and the battery finds three things, only one of which anybody was looking for.

8.1 The population stability index, and its two failure modes

For a covariate binned into levels with development shares p_ℓ and application shares q_ℓ, the index is

$$\mathrm{PSI} = \sum_{\ell} (q_\ell - p_\ell) \log \frac{q_\ell}{p_\ell},$$

which is the symmetric, or Jeffreys, form of the Kullback-Leibler divergence. The chi-square statistic on the same table asks a different question, namely whether the

difference is larger than sampling noise, and on samples of this size it answers yes to almost everything.

THE THRESHOLDS ARE CONTESTED, SO QUOTE BOTH

The bands usually given are 0.10 and 0.25, and they are not settled. The widely cited industry rule from Bhalla (2015) puts the upper band at 0.20, whereas Du Pisanie, Allison and Visagie (2022) derive 0.10 and 0.25 from a simulation harness for significance testing. The higher bound has the better statistical argument and the weaker claim to standard practice, and a model governance file should record which convention it uses rather than presenting either as received.

The second failure mode is structural. Where a level is present in one sample and absent from the other, $\log(q_\ell / p_\ell)$ diverges and the index is undefined. Implementations routinely floor the share at some small ε, which returns a large finite number that is a property of ε rather than of the data. Any such variable should be reported as an emptied level and read qualitatively.

```
CAT = ["Rating", "Country", "Education", "HomeOwnershipType",
       "NewCreditCustomer", "VerificationType"]
NUM = {"Age":          [17, 25, 35, 45, 55, 100],
       "IncomeTotal":  [-1, 500, 1000, 1500, 2500, 1e12],
       "LoanDuration": [0, 12, 24, 36, 48, 120],
       "Amount":       [-1, 500, 1500, 3000, 6000, 1e9]}

dev_pd = d.filter(pl.col("LoanDate") <  cut).to_pandas()
oot_pd = d.filter(pl.col("LoanDate") >= cut).to_pandas()
print(f"development {len(dev_pd):,}, out-of-time {len(oot_pd):,}, cut
        {cut.date()}")

def as_levels(s):
    return s.astype("Int64").astype(str) if
        pd.api.types.is_numeric_dtype(s) else s.astype(str)

def psi_chi(a, b):
    """PSI, chi-square, and the levels absent from one side."""
    pa, pb = a.value_counts(normalize=True),
        b.value_counts(normalize=True)
    lv = sorted(set(pa.index) | set(pb.index))
    p = np.array([pa.get(k, 0.0) for k in lv])
    q = np.array([pb.get(k, 0.0) for k in lv])
    absent = [lv[i] for i in range(len(lv)) if p[i] == 0 or q[i] == 0]
    eps = 1e-6
    pc, qc = np.clip(p, eps, None), np.clip(q, eps, None)
    psi = float(((qc - pc) * np.log(qc / pc)).sum())
    expected = pc * len(b)
    observed = np.array([(b == k).sum() for k in lv], float)
    chi = float(((observed - expected) ** 2 / expected).sum())
    return psi, chi, len(lv) - 1, absent

rows = [(v,) + psi_chi(as_levels(dev_pd[v]), as_levels(oot_pd[v])) for v
        in CAT]
rows += [(v,) + psi_chi(pd.cut(dev_pd[v], e,
        include_lowest=True).astype(str),
                        pd.cut(oot_pd[v], e,
        include_lowest=True).astype(str))
         for v, e in NUM.items()]

print(f"\n{'variable':<20}{'PSI':>9}{'chi2':>11}{'dof':>5}   reading")
for v, psi, chi, dof, absent in sorted(rows, key=lambda r: -r[1]):
    reading = (f"undefined, levels absent: {absent}" if absent else
               "material" if psi >= 0.25 else
               "investigate" if psi >= 0.10 else "stable")
    print(f"{v:<20}{psi:9.4f}{chi:11.1f}{dof:5d}   {reading}")
```

```
development 118,859, out-of-time 29,874, cut 2019-11-04

variable                 PSI       chi2  dof   reading
VerificationType      1.2616     9317.4    4   undefined, levels absent:
['0', '2', '3']
HomeOwnershipType     0.4547     4799.8   11   undefined, levels absent:
['-1', '0']
Rating                0.3521    11115.0    7   material
Education             0.2699     4246.7    6   undefined, levels absent:
['-1', '0']
Amount                0.1153     3596.3    4   investigate
LoanDuration          0.0561     1346.7    4   stable
Country               0.0227      174.6    3   undefined, levels absent:
['SK']
IncomeTotal           0.0151      460.7    4   stable
Age                   0.0027       81.6    4   stable
NewCreditCustomer     0.0024       72.3    1   stable
```

Four of the ten covariates have an emptied level, so four of the ten indices are artefacts of the epsilon floor rather than measurements. Of the six that are readable, `Rating` at 0.3521 is material on either convention, `Amount` at 0.1153 sits in the investigate band, and the remaining four are stable, with `Age` at 0.0027 and `NewCreditCustomer` at 0.0024 essentially identical across the two samples.

Note what has happened to `VerificationType`, which is a covariate in the generalised linear model of lecture 2 and in every network fitted since. The development sample carries levels 0 through 4 and the out-of-time sample carries only 1 and 4. The model therefore holds coefficients for three levels that the application window never presents, which is a milder problem than the reverse and still a fact a validator would want stated.

8.2 The diagnostic finds a data break before it finds any drift

Two Bondora fields behave in a way that no drift statistic should be asked to summarise.

```
breaks = (dat.with_columns(u=month_index("LoanDate"))
             .group_by("u")
             .agg(n=pl.len(),
                  dti_zero=(pl.col("DebtToIncome") == 0).mean(),
                  use_missing=(pl.col("UseOfLoan") == -1).mean())
             .sort("u"))
prev = None
for r in breaks.iter_rows(named=True):
    if prev is not None:                       # report the death only,
        not the birth
        if prev["dti_zero"] < 0.99 <= r["dti_zero"]:
            print(f"DebtToIncome becomes uniformly zero from
        {as_month(r['u'])}")
        if prev["use_missing"] < 0.99 <= r["use_missing"]:
            print(f"UseOfLoan becomes uniformly -1 from
        {as_month(r['u'])}")
    prev = r
tail = breaks.filter(pl.col("u") >= 2017 * 12 + 6)
print(f"from July 2017 onward: {tail['n'].sum():,} loans, "
      f"DebtToIncome zero in {(tail['n'] * tail['dti_zero']).sum() /
        tail['n'].sum():.1%}, "
      f"UseOfLoan missing in {(tail['n'] * tail['use_missing']).sum() /
        tail['n'].sum():.1%}")
```

```
DebtToIncome becomes uniformly zero from 2017-07
UseOfLoan becomes uniformly -1 from 2017-07
from July 2017 onward: 110,816 loans, DebtToIncome zero in 100.0%,
UseOfLoan missing in 100.0%
```

From July 2017 the platform stops populating `DebtToIncome` and `UseOfLoan` entirely. Every loan from that month onward carries a debt-to-income ratio of exactly zero and a loan purpose of -1, and both persist to the end of the extract. Consequently the development sample is itself two populations spliced together, and the splice predates the out-of-time cutoff by more than two years. That is a data-collection break rather than a population shift, and the third eligibility rule in section 5 exists precisely for it.

The series escaped consequences by luck. Neither field appears in any fitted model: lecture 1 lists `DebtToIncome` in its data dictionary and no lecture uses it as a covariate. Had lecture 2 included it in GLM3, the coefficient would have been estimated on pre-2017 loans alone and then applied to a test sample in which the variable is a constant,

and no in-sample diagnostic would have shown anything wrong. A representativeness sweep is the cheapest place to catch this, and the series had never run one.

8.3 The joint distribution, which the marginals miss

Marginal indices test one covariate at a time. A shift in which each margin looks stable while the dependence between covariates has moved will pass every one of them. The classifier two-sample test asks the question directly: pool the two samples, label each row by which sample it came from, and train a classifier to tell them apart. Where the samples are exchangeable the classifier cannot beat chance and its area under the curve tends to 0.5.

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_predict, StratifiedKFold
from sklearn.metrics import roc_auc_score
from sklearn.inspection import permutation_importance

C2 = pd.concat([dev_pd, oot_pd])[CAT + list(NUM)].copy()
for c in CAT:
    C2[c] = as_levels(C2[c]).astype("category")
origin = np.r_[np.zeros(len(dev_pd)), np.ones(len(oot_pd))]

clf = HistGradientBoostingClassifier(categorical_features=CAT,
        max_iter=120, random_state=1)
oof = cross_val_predict(clf, C2, origin, method="predict_proba",
                        cv=StratifiedKFold(5, shuffle=True,
        random_state=1))[:, 1]
print(f"C2ST cross-validated AUC = {roc_auc_score(origin, oof):.4f}   "
      f"(0.5 would mean the two samples are indistinguishable)")

clf.fit(C2, origin)
imp = permutation_importance(clf, C2, origin, n_repeats=5,
                             random_state=1, scoring="roc_auc")
print("\ndrop in AUC when each covariate is permuted:")
for i in np.argsort(-imp.importances_mean):
    print(f"   {C2.columns[i]:<20}{imp.importances_mean[i]:+.4f}")
```

```
C2ST cross-validated AUC = 0.9595   (0.5 would mean the two samples are
indistinguishable)

drop in AUC when each covariate is permuted:
   Amount              +0.2417
   Rating              +0.1081
   VerificationType    +0.0694
   Country             +0.0344
   HomeOwnershipType   +0.0246
   NewCreditCustomer   +0.0123
   Education           +0.0120
   LoanDuration        +0.0021
   IncomeTotal         +0.0021
   Age                 +0.0020
```

The classifier separates the development sample from the out-of-time sample at an area under the curve of 0.9595. Given a loan and its ten covariates, a model can say which side of November 2019 it was written on almost every time.

Set that against the marginal table. Six of the ten covariates are readable there and four of those six are stable, with three sitting below a population stability index of 0.03. The two readings are not in conflict, and the resolution is the point of the exercise: the margins have moved modestly while the joint distribution has moved enormously, and only the second is visible to a test that looks at the covariates together.

The importance ranking sharpens it further. `Amount` is by a wide margin the strongest separator, costing 0.24 of area under the curve when permuted, yet its population stability index is 0.1153, which the conventional bands classify as worth investigating rather than material. The marginal index and the joint test disagree about which variable matters most, and on this book the joint test is the one telling the truth.

> **WHAT THE CLASSIFIER TEST IS, AND IS NOT**
>
> The test is a complement to the stability index and not a replacement for it. The index monitors one covariate through time, which is what a monitoring pack needs, and it has threshold conventions a supervisor recognises. The classifier answers a single yes-or-no question about the joint distribution and locates the drivers, which is what a development sample needs before anyone fits anything.
>
> Two cautions. A high area under the curve says the samples differ and says nothing about whether the difference harms the model, so pair it with an impact analysis rather than reading it as a verdict. And the method reaches this series from an industry presentation rather than a peer-reviewed source, so the finding above stands on the computation shown here rather than on the authority of the reference.

9 Class imbalance

Lecture 1 recorded that Bondora defaults at 28.95 per cent, observed that rarity is a property of the portfolio rather than of credit risk, and promised that imbalance would return with force once the credit card portfolio took the stage. Lecture 3 put that

portfolio on the stage and named its 1.42 per cent bad rate without treating it. Here is the debt.

One guard before starting. The card file carries no origination date, so everything in this section is a statement about imbalance alone. Consequently the vintage argument of sections 6 and 7 does not transfer, and neither does the out-of-time reasoning that rests on it.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, average_precision_score

rng = np.random.default_rng(1)
card = pl.read_parquet("../data/credit_card_dev.parquet")
y = card["bad_flag"].to_numpy().astype(float)
feat = [c for c, t in zip(card.columns, card.dtypes)
        if t != pl.String and c not in ("account_number", "bad_flag")]
Xc = card.select(feat).to_numpy().astype(np.float32)
print(f"{len(y):,} accounts, {len(feat):,} numeric covariates, bad rate
        {y.mean():.4%}")
print(f"Bondora, for contrast: {dat['default_12m'].mean():.4%}")

perm = rng.permutation(len(Xc))
n_learn = int(0.8 * len(Xc))
learn, test = perm[:n_learn], perm[n_learn:]

med = np.nan_to_num(np.nanmedian(Xc[learn], axis=0))
Xc = np.where(np.isnan(Xc), med, Xc)
mu, sd = Xc[learn].mean(0), Xc[learn].std(0)
keep = sd > 0
Xc = np.clip((Xc[:, keep] - mu[keep]) / sd[keep], -10, 10)

corr = np.abs(np.corrcoef(np.c_[Xc[learn], y[learn]], rowvar=False)[-1,
        :-1])
sel = np.argsort(-np.nan_to_num(corr))[:40]          # ranked on the
        learning sample alone
Xt, Xs, yt, ys = Xc[learn][:, sel], Xc[test][:, sel], y[learn], y[test]
print(f"learn {len(yt):,} ({yt.mean():.4%} bad), test {len(ys):,}
        ({ys.mean():.4%} bad)")
```

```
96,806 accounts, 1,212 numeric covariates, bad rate 1.4173%
Bondora, for contrast: 28.9492%
learn 77,444 (1.4372% bad), test 19,362 (1.3377% bad)
```

Three fits follow, all on the same forty covariates: one with no correction, one on a one-to-one random undersample of the majority class, and one on a one-to-one random oversample of the minority.

```
def assess(name, model, offset=0.0):
    lp = model.decision_function(Xs) + offset
    p = 1 / (1 + np.exp(-lp))
    print(f"{name:<26} AUC {roc_auc_score(ys, p):.4f}   "
          f"PR-AUC {average_precision_score(ys, p):.4f}   "
          f"mean predicted {p.mean():.4%}   observed {ys.mean():.4%}   "
          f"ratio {p.mean() / ys.mean():.2f}")

base = LogisticRegression(max_iter=2000).fit(Xt, yt)
assess("no correction", base)

bad = np.where(yt == 1)[0]
good = np.where(yt == 0)[0]
under = np.r_[rng.choice(good, size=len(bad), replace=False), bad]
m_under = LogisticRegression(max_iter=2000).fit(Xt[under], yt[under])
assess("undersampled to 1:1", m_under)

over = np.r_[good, rng.choice(bad, size=len(good), replace=True)]
m_over = LogisticRegression(max_iter=2000).fit(Xt[over], yt[over])
assess("oversampled to 1:1", m_over)
```

```
no correction              AUC 0.7945   PR-AUC 0.0563   mean predicted
1.4333%   observed 1.3377%   ratio 1.07
undersampled to 1:1        AUC 0.7891   PR-AUC 0.0509   mean predicted
38.2375%   observed 1.3377%   ratio 28.59
oversampled to 1:1         AUC 0.7932   PR-AUC 0.0531   mean predicted
38.1703%   observed 1.3377%   ratio 28.53
```

The uncorrected fit is well calibrated, predicting 1.43 per cent against an observed 1.34 per cent. Both corrected fits predict roughly 38 per cent on a book that defaults at 1.34, which is an overstatement by a factor of about 28. Neither buys any discrimination: the area under the curve is 0.7945 uncorrected, 0.7891 undersampled and 0.7932 oversampled, so the corrections range from neutral to mildly harmful. The precision-recall area moves the same way, from 0.0563 down to 0.0509.

That is exactly the result van den Goorbergh, van Smeden, Timmerman and Van Calster reported in 2022 across simulations and a clinical case study. Their abstract puts it plainly: all correction methods "led to poor calibration (strong overestimation of the probability to belong to the minority class), but not to better discrimination", and their conclusion is that "outcome imbalance is not a problem in itself, and that imbalance correction may even worsen model performance".

9.1 The balance property survives, on a measure nobody wanted

Lecture 7 owns the balance property, so this section states it exactly rather than loosely. The property is an identity of the fitted model on its own fitting sample, and a resampled or weighted fit satisfies it there in full.

```
for nm, model, idx in (("uncorrected", base, np.arange(len(yt))),
                       ("undersampled", m_under, under)):
    fitted = model.predict_proba(Xt[idx])[:, 1]
    print(f"{nm:<13} on its own fitting sample: "
          f"observed {yt[idx].sum():.0f} bads, fitted
        {fitted.sum():.1f}")

print("\non the portfolio learning sample:")
print(f"   observed                {yt.sum():.0f} bads")
print(f"   uncorrected model       {base.predict_proba(Xt)[:,
        1].sum():.0f}")
print(f"   undersampled model      {m_under.predict_proba(Xt)[:,
        1].sum():.0f}")
```

```
uncorrected   on its own fitting sample: observed 1113 bads, fitted
1108.8
undersampled  on its own fitting sample: observed 1113 bads, fitted
1112.8

on the portfolio learning sample:
   observed                1113 bads
   uncorrected model       1109
   undersampled model      29733
```

Each model reproduces the observed count on the sample it was fitted to, to within rounding. Applied to the portfolio, the undersampled model predicts 29,733 defaults where 1,113 occurred. The identity held throughout, and the measure it held on stopped being the portfolio the moment the majority class was thinned.

9.2 Putting the level back

Where undersampling is unavoidable, the intercept can be corrected analytically. With a population prevalence τ and a sample prevalence $\bar{y}$, the correction is

$$\hat{\beta}_0^{\,\text{corrected}} = \hat{\beta}_0 - \log\!\left(\frac{1-\tau}{\tau}\cdot\frac{\bar{y}}{1-\bar{y}}\right),$$

which is the standard prior correction for choice-based sampling.

```
tau, ybar = yt.mean(), 0.5
offset = -np.log(((1 - tau) / tau) * (ybar / (1 - ybar)))
print(f"population prevalence {tau:.4%}, sample prevalence {ybar:.0%}, "
      f"offset {offset:.4f}")
assess("undersampled + offset", m_under, offset=offset)
```

```
population prevalence 1.4372%, sample prevalence 50%, offset -4.2280
undersampled + offset      AUC 0.7891   PR-AUC 0.0509   mean predicted
1.7842%   observed 1.3377%   ratio 1.33
```

The correction moves the mean prediction from 38.24 per cent to 1.78 per cent against an observed 1.34, so it recovers the order of magnitude and leaves a 33 per cent overstatement behind. It changes no ranking at all, since adding a constant to every linear predictor leaves the area under the curve at 0.7891 exactly. The honest summary is that undersampling costs calibration, that the offset returns most of it, and that not undersampling costs nothing and returns all of it.

Botha and Verster take the third road, and it is worth stating because it is neither of the two above. They fit a weighted logistic regression with defaulted observations weighted ten to one, and they apply no intercept correction whatever, because they chose the

weight by minimising the error against the empirical twelve-month default rate. The calibration is imposed by construction rather than repaired afterwards.

9.3 Which metrics survive a 1.4 per cent base rate

Lecture 2 established that the area under the curve, accuracy and F_1 are invariant under any monotone rescaling of the scores, so they leave the level of PD_k unidentified. Under heavy imbalance a second problem arrives on top of the first.

Saito and Rehmsmeier (2015) show that the visual interpretability of a receiver operating characteristic plot "can be deceptive with respect to conclusions about the reliability of classification performance, owing to an intuitive but wrong interpretation of specificity". The mechanism is that a false positive rate is measured against a very large negative class, so a specificity of 0.99 still admits an enormous number of false positives in absolute terms. A precision-recall plot escapes it because precision evaluates "the fraction of true positives among positive predictions", which is the quantity an operational cutoff actually delivers.

The numbers above show the gap. The uncorrected model reaches an area under the receiver operating characteristic curve of 0.7945, which reads as a respectable model. Its precision-recall area is 0.0563 against a no-skill baseline of 0.0134, so it is roughly four times better than random at the thing a collections queue cares about. Both statements describe the same fitted model.

10 What a validator asks, and what this lecture does not show

The questions this lecture equips a reader to answer are the ones a validator opens with. Which snapshots were eligible, and on what rule? Was the split drawn on loans or on rows? Is the out-of-time sample representative of the application population on the

margins, and on the joint distribution? Which periods were excluded, on what evidence, and what did the exclusion move? Was the response rebalanced, and if so what put the level back?

Four limitations bound everything above.

Bondora is not an internal ratings-based portfolio, and never will be. Every regulatory reference here illustrates mechanics on a book that would not qualify. The point is the procedure rather than the number.

The relief window is dated from the platform's own rescheduling field, which is a good proxy on this book and is not what a bank would use. A lender has the forbearance flag, the moratorium indicator and the collections policy calendar, and would date the window from those.

The exclusion demonstration uses one specification. A baseline hazard in age bands with country and rating is enough to show the direction and the order of magnitude of the effect, and a production model with time-varying covariates could move the 2.37 per cent either way.

The imbalance section is cross-sectional throughout. The card portfolio has no origination date, so nothing there speaks to how an imbalanced response interacts with vintage drift, which is the combination most retail books actually present.

Lecture C2, already named and deferred, takes attribution into the causal frame. The natural sequel to this lecture is the interaction between resampling and the encodings of lecture 6, since a weight of evidence table estimated on a rebalanced sample carries the rebalancing into every downstream fit.

11 References

Baesens, B., Roesch, D., and Scheule, H. (2016). *Credit Risk Analytics: Measurement Techniques, Applications, and Examples in SAS*. Wiley.

Bhalla, D. (2015). *Population Stability Index and Characteristic Analysis*. ListenData.

Botha, A., and Verster, T. (2025). *Approaches for modelling the term-structure of default risk under IFRS 9: a tutorial using discrete-time survival analysis*.

Botha, A., Verster, T., and Scheepers, B. (2025). *Exploring different subtypes of recurrent event Cox-regression models in modelling lifetime default risk: a tutorial*. arXiv:2505.01044.

Coelho, R., and Zamil, R. (2020). *Payment holidays in the age of Covid: implications for loan valuations, market trust and financial stability*. FSI Briefs No 8, Bank for International Settlements.

Djurovic, A. (2025). *Rethinking Representativeness Analysis in IRB Modeling: Classifier Two-Sample Test*.

Du Pisanie, J., Allison, J. S., and Visagie, J. (2022). *A proposed simulation technique for population stability testing in credit risk scorecards*. arXiv:2206.11344.

European Parliament and Council (2013). *Regulation (EU) No 575/2013 on prudential requirements for credit institutions and investment firms*, Articles 174, 175 and 180.

Prudential Regulation Authority (2026). *SS4/24: Credit risk: internal ratings based approach*, January 2026 consolidation, sections 8 and 11.

Saito, T., and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.

van den Goorbergh, R., van Smeden, M., Timmerman, D., and Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association*, 29(9), 1525-1534.